

Battery State of Charge Agentic AI Workshop

Participant Preparation and Hands On Guide

This guide prepares you for a one-hour MATLAB R2026a workshop on a supported Windows computer or Apple-silicon Mac. The workflow begins with a read-only environment preflight, uses prepared PyTorch reference data without requiring an external Python environment, validates a MATLAB-native network in Simulink, and ends with host MEX verification and generated C-code inspection. Participants without a configured compiler can complete the modeling workflow using the prepared MEX evidence.

Workshop outcome

- Verify MATLAB, MCP connectivity, required products, and the embedded AI skill before beginning.
- Evaluate a prepared MATLAB-native LSTM State-of-Charge estimator.
- Import the PyTorch MLP as a MATLAB-native dlnetwork and validate it against the prepared original-PyTorch reference outputs.
- Compare the baseline LSTM with the projected and fine-tuned LSTM checkpoint.
- Compare MATLAB, Simulink Normal Simulation, desktop MEX, and generated C-code evidence in gated phases.

Model path through the workshop

Part	Participant activity	Model used	Result
1	Evaluate the prepared MATLAB model	MATLAB-native LSTM	Baseline SOC accuracy
2	Import and verify a PyTorch model	PyTorch MLP rebuilt as dlnetwork	Framework equivalence
3	Validate system integration	MATLAB-native AI Simulink model	MATLAB versus Simulink evidence
4	Compare compression and generate code	Projected and fine-tuned LSTM	MEX and C-code evidence

Complete before the event

Complete this checklist before arriving. Installation during the workshop is a recovery option and may be blocked by company policy.

- ☐ Bring a supported Windows laptop or Apple-silicon Mac and its charger. MATLAB R2026a is not available for Intel-based Macs.
- ☐ Confirm internet access to GitHub, MathWorks, and the official service used by your selected AI agent. Windows participants may also need Microsoft Store access; macOS participants may need Apple App Store access for Xcode.
- ☐ Confirm that you can sign in to MATLAB and to one AI coding agent: Codex, Claude Code, or Google Antigravity CLI. Use an existing paid subscription if available; otherwise a free tier may be used subject to its allowance.
- ☐ Obtain IT approval for local software, PowerShell or Terminal commands, Git, MATLAB Add-Ons, and a supported C/C++ compiler.
- ☐ Install MATLAB R2026a and the required products below.
- ☐ Install only one AI coding agent. Codex is the workshop-validated default; Claude Code is a supported alternative; Antigravity CLI is an optional advanced alternative.
- ☐ Download or clone the workshop repository. (You can also do this with the instructor during the workshop.)
- ☐ Open Aimbdworkshop.prj once before the event. (You can also do this with the instructor during the workshop.)

Required MATLAB products

Type	Required item	Used for
Release	MATLAB R2026a	Agent Skill workflow
Product	Simulink	System integration
Product	Deep Learning Toolbox	LSTM and imported network
Product	Statistics and Machine Learning Toolbox	Project workflow dependency
Product	MATLAB Coder	Host MEX and C code
Product	Simulink Coder	Simulink code generation
Product	Embedded Coder	Embedded C code and reports
Product	Fixed-Point Designer	Embedded numeric workflows
Add-On	Deep Learning Toolbox Model Compression Library	Network projection
Add-On	MATLAB Coder Interface for Deep Learning	Code generation from dlnetwork
Add-On	Deep Learning Toolbox Converter for PyTorch Models	Part 2 model import
Compiler	Windows: MinGW-w64 or supported Microsoft compiler macOS: Xcode 16 or Xcode 26	Desktop MEX verification

Python and PyTorch

External Python and PyTorch are not required for the guided workshop. The Deep Learning Toolbox Converter for PyTorch Models add-on is still required to import `mlp_soc_model.pt2`. Part 2 uses the prepared inputs together with `agentic_ai/reference_data/part2_pytorch_reference_outputs.csv`, which contains 200 outputs generated by the original PyTorch model in the same sample order. Do not create a Python environment or install Python packages during the session.

Choose one AI coding agent

Option A ChatGPT Desktop with Codex

Install the ChatGPT desktop app for your operating system before the event from <https://chatgpt.com/download>. Windows participants may use the Microsoft Store or the following Windows Package Manager command when company policy permits it.

```
winget install --id 9PLM9XGG6VKS -s msstore
```

Official ChatGPT desktop app download

ChatGPT account and Codex usage

A ChatGPT Free account can be used for the guided exercises, and a paid plan is not required to attend. Codex usage is not determined only by the words in a prompt; it also varies with model choice, task complexity, context length, reasoning, and tool calls. A long end-to-end task can therefore use more allowance than several short gated tasks.

Before the event, sign in and run one short Codex test. Check the usage indicator in Codex or use `/status` in Codex CLI when available. If a limit is reached, continue with the instructor-prepared scripts and results. Do not share account credentials. An OpenAI API key is not required for this workshop.

Exact token totals are not a workshop requirement and may not be shown for a ChatGPT-plan Codex session. The workshop therefore uses short phase prompts, approval gates, and prepared fallback evidence.

Option A Codex CLI fallback on Windows

The PowerShell installer below is a Windows fallback. On macOS, use the ChatGPT desktop app for this workshop unless your IT team has approved a separate Codex CLI installation.

```
powershell -ExecutionPolicy Bypass -c "irm https://chatgpt.com/codex/install.ps1 | iex"
```

The command uses the official OpenAI installer. ExecutionPolicy Bypass applies only to the PowerShell process; it does not override company security controls. If installation is blocked, pair with another participant or follow along with the instructor.

```
codex
```

At first launch, select Sign in with ChatGPT and complete authentication.

Option B Claude Code CLI

Claude Code is supported by the MATLAB Agentic Toolkit and can run the same gated workshop prompts after MATLAB MCP and the embedded AI skill are available. Install it only if you prefer Claude Code instead of Codex.

Windows PowerShell

```
irm https://claude.ai/install.ps1 | iex
```

macOS Terminal

```
curl -fsSL https://claude.ai/install.sh | bash
```

Verify the installation with `claude --version`, change to the workshop repository, and start the agent with `claude`. Sign in when prompted. `setupAgenticToolkit("install")` can configure supported Claude Code integration. In Claude Code, use `/skills` to confirm the MATLAB and embedded AI skills are visible before starting Part 1.

Option C Google Antigravity CLI

Antigravity CLI supports MCP servers and Agent Skills, but this is an optional advanced route and is not the workshop-validated baseline. Manual MATLAB MCP and skill configuration may be required. Choose it only if you already use Antigravity or can complete the preflight before the event.

Windows PowerShell

```
irm https://antigravity.google/cli/install.ps1 | iex
```

macOS Terminal

```
curl -fsSL https://antigravity.google/cli/install.sh | bash
```

Start the CLI with `agy`. Use `/mcp` to confirm the MATLAB MCP server is connected and `/skills` to confirm that the embedded AI skill is loaded. For a repository-local installation, place the skill folder under `.agents/skills/embedded-ai-deployment`. If either check fails, use Codex or Claude Code for the hands-on session.

Selection guidance

Install only one agent. Use Codex for the closest match to the tested workshop flow. Use Claude Code when your organization already approves Anthropic. Use Antigravity only after its MATLAB connection passes preflight. The engineering prompts are the same; the agent must be connected to the live MATLAB session and must load `embedded-ai-deployment` or `matlab-deploy-embedded-ai`.

Download the lab files

Clone the repository before the event when possible.

```
git clone https://github.com/KantikaAscendas/Agentic-AI-Embedded-Skill-Workshop.git
cd Agentic-AI-Embedded-Skill-Workshop
```

[Agentic AI Embedded Skill Workshop repository](#)

If Git is unavailable, use Code then Download ZIP on GitHub, extract the ZIP to a short writable path, and avoid working directly inside the ZIP file.

Install the MATLAB Agentic Toolkit

1. Open the MATLAB Agentic Toolkit repository and download agenticToolkitInstaller.mltbx.
2. Open the downloaded MLTBX file with MATLAB and approve the add-on installation.
3. In MATLAB, run the installer command below.

```
setupAgenticToolkit("install")
```

4. Select only the groups needed for this workshop: MATLAB Core, AI and Statistics, and Code Generation. Include the Simulink Agentic Toolkit because the workshop uses Simulink models.
5. For every new MATLAB session, initialize the Simulink Agentic Toolkit with the operating-system-neutral commands below.

```
if ispc
    homeFolder = getenv("USERPROFILE");
else
    homeFolder = getenv("HOME");
end
addpath(fullfile(homeFolder, ".matlab", "agentic-toolkits", "simulink"))
satk_initialize
```

The satk_initialize command adds the Simulink tools, validates the installation, and shares the current MATLAB session with the MCP server. You do not need to run shareMATLABSession separately after successful initialization.

Install the embedded AI skill

Open your selected AI agent in the workshop repository and send the following prompt. Review and approve the requested installation. Start a new agent session after installation so the skill becomes available.

```
Install the embedded-ai-deployment skill from https://github.com/matlab/agent-skills-playground/tree/main/demos/embedded-ai-deployment/skills/embedded-ai-deployment
```

This demo-bundled package keeps the skill name embedded-ai-deployment, which is the version used to validate this workshop. New MATLAB Agentic Toolkit installations may also provide the official skill matlab-deploy-embedded-ai. The preflight script detects both names, prefers the validated demo skill when both are present, and prints the exact ACTIVE_SKILL to use in the remaining prompts.

[Agent Skills Playground embedded AI deployment skill](#)

Verify the setup

- Open MATLAB R2026a and Aimbdworkshop.prj.
- Run the operating-system-neutral addpath and satk_initialize commands shown above once in the current MATLAB session.
- Run agentic_ai/scripts/workshopPreflight.m and confirm that the final line is READY=true.
- If READY=false, keep the missing item visible and ask the instructor before continuing.
- Open your selected AI agent in the workshop repository and send the read-only preflight prompt below.

Use embedded-ai-deployment. If that skill is unavailable but matlab-deploy-embedded-ai is installed, use matlab-deploy-embedded-ai instead. Perform a read-only workshop preflight only. Do not inspect project data or models and do not modify files. Verify that a compatible embedded AI deployment skill is available, a live MATLAB MCP session is connected, the exact release is MATLAB

R2026a, required products and support packages are present, Simulink Agentic Toolkit tools are available, and `agentic_ai/scripts/workshopPreflight.m` passes. Report the exact `ACTIVE_SKILL` printed by the script. Detect the host operating system and processor architecture. On macOS, confirm Apple silicon and a selected Xcode 16 or Xcode 26 C and C++ compiler. Report Python separately as managed runtime available, external Python available, or unavailable. Confirm that `agentic_ai/generated` and `agentic_ai/results` are writable. If the compiler is missing but the other requirements pass, return `READY WITH FALLBACK` and state that desktop MEX compilation will use instructor-prepared evidence. Return a concise `PASS`, `WARNING`, or `FAIL` table and finish with `READY`, `READY WITH FALLBACK`, or `NOT READY`. Stop and wait for approval.

Workshop workflow

Time	Activity	Participant evidence
0 to 8	Initialize MATLAB and run the read-only preflight	Readiness table
8 to 18	Part 1 baseline LSTM evaluation	SOC prediction and RMSE
18 to 32	Part 2 PyTorch MLP import and equivalence	Native dlnetwork and reference comparison
32 to 42	MATLAB versus Simulink comparison	Normal Simulation equivalence
42 to 55	Part 3 live projection, prepared fine-tuned comparison, and host code verification	Size, RMSE delta, MEX result, C report
55 to 60	Review and questions	Evidence summary and next steps

Start prompt

Use the `ACTIVE_SKILL` reported by `agentic_ai/scripts/workshopPreflight.m`: `embedded-ai-deployment` for the validated demo package, or `matlab-deploy-embedded-ai` when only the official MATLAB Agentic Toolkit skill is installed. Work with this battery State-of-Charge repository in the connected MATLAB R2026a session. Preserve `Exercise_1.m`, `Exercise_2.m`, and `Exercise_3.m`. Create executable `.m` scripts under `agentic_ai/generated` and results under `agentic_ai/results`; also maintain a participant-facing Live Script summary when supported. Work one phase at a time, explain the evidence, and pause for approval. The participant workflow ends after host verification and C-code inspection. Begin with project discovery and a concise project summary only.

Part 1 prompt

Proceed with the prepared data split and baseline MATLAB-native LSTM evaluation. Do not retrain. Use the existing checkpoint and a representative subset of held-out test data. Report the test count and RMSE, save a prediction plot, then pause.

Part 2 prompt

Proceed with Part 2 using `Part_2_AI_import/models/mlp_soc_model.pt2`. Import it with the modern MATLAB R2026a workflow and rebuild it as a MATLAB-native `dlnetwork` suitable for the Cortex-M deployment pattern. Do not create or configure an external Python environment. Use the prepared inputs and `agentic_ai/reference_data/part2_pytorch_reference_outputs.csv` as the

original PyTorch reference, and keep the prepared sample order unchanged. Before equivalence testing, propose 20 representative cases, using five evenly spaced samples from each of the four temperature groups, and wait for approval. After approval, report MAE, RMSE, maximum absolute error, and cosine similarity. Open the native network in Deep Network Designer, run the existing Simulink integration model, save the comparison evidence, then pause. Do not treat this MLP as the final embedded model.

When the agent proposes the 20 cases, send:

Approved. Proceed with the 20-case equivalence test and Simulink integration.

Validated reference result: MAE 6.26e-8, RMSE 1.05e-7, maximum absolute error 2.38e-7, and cosine similarity 1.0. Small last-digit differences are acceptable. The MLP demonstrates framework import and integration; the projected and fine-tuned LSTM remains the final embedded model in Part 3.

Why Simulink comparison

Simulink validates system-level integration before code generation: signal dimensions, single-precision datatypes, sample time, and output agreement. The required comparison uses the prepared MATLAB-native network. A masked direct-PyTorch Predict or S-function block is optional because R2026a size or datatype propagation can fail even when the imported network is numerically correct.

Simulink comparison prompt

Proceed with the prepared MATLAB-native AI Simulink model. Use the same single-precision inputs for MATLAB prediction and Simulink Normal Simulation. Report input size, datatype, sample time, RMSE, and maximum absolute error. If a direct PyTorch Simulink block fails size or datatype propagation, record an optional-path warning and continue with the MATLAB-native Simulink model without modifying the original exercise files. Save the comparison evidence, then pause.

Part 3 live compression prompt

Proceed with Part 3 using the embedded AI Pattern 1 workflow. The workshop compression decision is approved: the target is an ARM Cortex-M7 with Simulink, the primary goal is smaller flash and model size, single-precision floating point is acceptable, and no live retraining will be performed.

Create and execute a visible MATLAB script under `agentic_ai/generated`. Load the baseline LSTM from `Part_1_AI_modeling/models/trainedNetwork.mat`. Use the same representative training/calibration selection defined in `Part_3_Code_Gen/Exercise_3.m` to calculate neuron PCA. Call `compressNetworkUsingProjection` once with `LearnablesReductionGoal=0.95` to create one live projected candidate. Check the installed MATLAB R2026a help for the exact function signature before calling it. Do not run a compression sweep, do not call `trainnet`, do not modify the original exercise files, and do not overwrite `Part_3_Code_Gen/models/dlnetFineTuned.mat`.

Evaluate three networks: the baseline LSTM, the newly projected candidate before fine-tuning, and the prepared projected and fine-tuned checkpoint from `Part_3_Code_Gen/models/dlnetFineTuned.mat`. Run all three on exactly the same independent held-out test sequences used in Part 1, with the same sample order, sample count, and single-precision inputs. Report learnables, estimated parameter bytes, actual memory reduction, RMSE, MAE, maximum absolute error, and each error delta from the baseline. Save the live projected candidate, metrics table, and comparison plots under `agentic_ai/results`.

Explain explicitly that the projected candidate was compressed live by the Agent, while the prepared checkpoint is loaded only to demonstrate the accuracy recovery obtained previously through fine-tuning. Verify that the live projected candidate and prepared checkpoint have compatible projected architecture and learnable counts before describing them as before- and after-fine-tuning versions. If they differ, report the difference and treat them as separate compressed candidates.

After presenting the accuracy-memory trade-off, pause for approval before desktop MEX generation. Do not proceed to C code in this step.

Troubleshooting

Issue	Action
Company policy blocks installation	Do not bypass policy. Pair with a prepared participant or follow the instructor during the workshop.
Codex cannot see MATLAB	Run the operating-system-neutral addpath command and satk_initialize in the MATLAB session you want Codex to use, then retry.
Skill is not available	Install the demo-bundled embedded-ai-deployment package, start a new Codex task, rerun workshopPreflight.m, and use the exact name printed after ACTIVE_SKILL=. The official Toolkit alternative is matlab-deploy-embedded-ai.
Preflight reports a missing item	Install the named MATLAB product or Add-On before continuing.
MEX compiler is missing	Windows: install/configure MinGW-w64 or a supported Microsoft compiler. macOS: install Xcode 16 or 26, accept its license, then run mex -setup C and mex -setup C++. Continue as READY WITH FALLBACK if installation is blocked.
Build takes too long	Use the prepared checkpoint and instructor-generated artifacts; do not retrain.
OneDrive or antivirus slows builds	Use a short local writable project path for the workshop copy.
Python or PyTorch is unavailable	Continue normally with the prepared Part 2 inputs and 200-row PyTorch reference CSV. Do not create a Python environment or install Python packages during the workshop.
Direct PyTorch Simulink block fails	Record an optional-path warning and continue with the prepared MATLAB-native Simulink model.
Codex usage limit is reached	Continue with the generated scripts, prepared results, and instructor demonstration. Do not share credentials.

Reference links

- [Workshop repository](#)
- [MATLAB Agentic Toolkit](#)
- [Agent Skills Playground](#)
- [Codex CLI guide](#)
- [ChatGPT desktop app for Windows](#)
- [Official Codex pricing and usage guidance](#)

- MATLAB R2026a macOS system requirements:
<https://www.mathworks.com/content/dam/mathworks/mathworks-dot-com/support/sysreq/files/system-requirements-release-2026a-macintosh.pdf>
- MATLAB R2026a supported compilers:
<https://www.mathworks.com/content/dam/mathworks/mathworks-dot-com/support/sysreq/files/system-requirements-release-2026a-supported-compilers.pdf>